UM12403

EtherCAT® Industrial Fieldbus Protocol

Rev. 3.0 — 12 December 2025 **User manual**

Document information

Information	Content
Keywords	UM12403, EtherCAT, MIMXRT1180-EVK
Abstract	This document serves as user manual and quick start guide for EtherCAT Fieldbus protocol. It also describes EtherCAT library and APIs needed to port it with and without GOAL, creating sample applications and creating EtherCAT main device PLC using CODESYS. Supports: MIMXRT1180-EVK.

1 Introduction

EtherCAT is an Industrial Ethernet fieldbus. It uses standard IEEE 802.3 Ethernet frames to transport information across the network. Ethernet frames transporting EtherCAT datagrams use the registered EtherType 0x88A4. EtherCAT networks have a single main and multiple sub devices. The main device uses a standard MAC controller to send and receive EtherCAT frames. The sub devices require an EtherCAT Sub device controller (ESC).

1.1 Supported platforms

The EtherCAT library runs on targets with or without an OS. The CPU connects to an EtherCAT Sub device Controller (ESC). MIMXRT1180-EVK is the only NXP platform supporting GOAL EtherCAT.

1.2 EtherCAT sub device controller

All EtherCAT sub devices require an EtherCAT sub device controller. The controller processes EtherCAT frames from the main device on the fly by reading and writing data as the frame passes through the sub device, which enables low latency and low jitter processing. Each sub device forwards the frame to the next sub device, and the last sub device returns the frame to the main device. An ESC contains registers and Process Data RAM. The main device accesses these registers and RAM through EtherCAT frames. The CPU of the sub device uses the ESC Process Data Interface (PDI). The ESC ensures data consistency. The registers provide configuration and status information, and the Process Data RAM enables cyclic and acyclic data exchange between the main and sub devices.

1.3 Sync managers

ESC includes at least four sync managers. Sync managers guard areas of Process Data RAM and provide locking mechanisms for consistent process data. Sync managers also notify both main device and sub device about new data. Configure sync managers to operate in Buffered Mode or Mailbox Mode.

To transfer acyclic data for application layer protocols, you can use Mailbox Sync Managers. The main device can write to a mailbox and the sub device can read from it, or vice versa. Two mailboxes enable acyclic communication in both directions by locking the buffer for one side during access.

To transfer process data in real time, you can use Sync Managers in Buffered mode. This mode allows simultaneous reading and writing by reserving three buffers of equal size. Only the first buffer is visible to both sides. One buffer handles writing, one handles reading, and the third stores the last written data. This setup allows new data to be written while old data is still read. Typically, two sync managers in Buffered Mode transfer process data in both directions.

The EtherCAT specification defines required sync manager usage. Only input sub devices can use Sync Manager 2 as input.

Table 1. Sync Manager Usage

Sync Manager	Name	Direction	Main view	Sub view	PDO
0	Mailbox Write	M -> S	Output	Consumer	
1	Mailbox Read	S -> M	Input	Producer	
2	Process Data Write	M -> S	Output	Consumer	RxPDO
3	Process Data Read	S -> M	Input	Producer	TxPDO

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

1.4 Mailbox protocols

As stated above, the ESC usually provides two Mailboxes for acyclic data exchange. This section introduces the Application Layer Protocols transferred via mailbox.

1.4.1 CANopen over EtherCAT (CoE)

The CANopen over EtherCAT (CoE) protocol enables access to a CANopen object dictionary. CANopen serves as a common fieldbus based on CAN. CoE organizes acyclic data in an object dictionary divided into different areas.

Table 2. CoE object dictionary

Index range	Area
0x0000 - 0x0FFF	Data Type Area
0x1000 - 0x1FFF	CoE Communication Area
0x2000 - 0x5FFF	Manufacturer-Specific Area
0x6000 - 0xFFFF	Profile Area

CoE provides services to upload and download objects, called Service Data Objects (SDOs), similar to CANopen. The sub device can send an emergency message to the main device to report local errors. CoE messages support more efficient data transfer because they are not limited to 8 bytes, like CANopen. The SDO Info Service enables the main device to read all meta information about objects. CoE introduces Complete Access, which uploads or downloads all subindices of an object at once. Segmented SDO transfer handles objects that do not fit into a mailbox.

Special objects define PDO mapping. A Process Data Object (PDO) specifies which objects transfer as cyclic process data. PDO-mapping objects contain entries that define the index, subindex, and bit size for process data.

Table 3. Table: PDO-mapping objects

Index range	PDO	Direction
0x1600 - 0x17FF	RxPDO	M -> S
0x1A00 - 0x1B00	TxPDO	S -> M

Each Sync Manager has a PDO Assignment Object (0x1C10 - 0x1C2F). Its entries list the PDO-Mapping objects that are assigned to a Sync Manager.

Table 4. Sync Manager Assignment objects

Sync Manager	Name	Direction	PDO Assignment Object
2	Process Data Write	M -> S	0x1C12
3	Process Data Read	S -> M	0x1C13

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

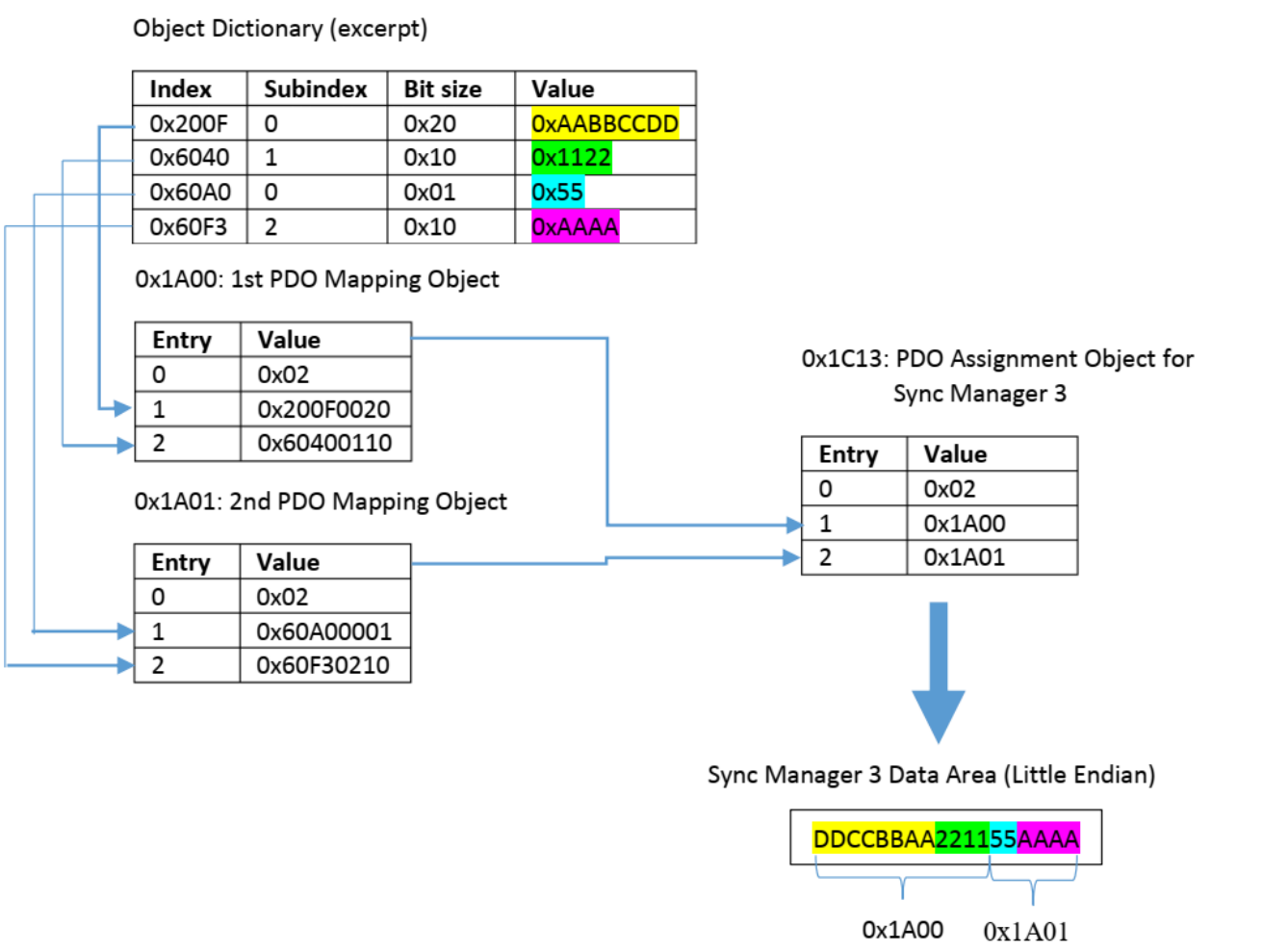

Figure 1. DO Mapping in EtherCAT

PDO Assignment objects and PDO-Mapping objects can be used to define the byte stream transferred by Process Data Sync Managers. Each PDO-Mapping object should be mapped sequentially to the byte stream as specified by the PDO Assignment object, without alignment. The byte stream starts with objects from the first assigned PDO-Mapping object, followed by objects from the next, and so on. Depending on direction, map objects from the Object Dictionary to the byte stream (TxPDO) or from the byte stream to the Object Dictionary (RxPDO).

The PDO mapping can be configured as static or dynamic. Static mapping does not allow changes to PDO Mapping and Assignment objects. Dynamic mapping allows changes to at least the PDO Assignment objects, enabling use of different predefined PDO-Mapping objects. Writable PDO-Mapping objects allow selection and order of process data objects as needed.

1.4.2 Ethernet over EtherCAT (EoE)

The EoE mailbox protocol allows the transfer of normal Ethernet frames via EtherCAT. The frames are fragmented and transferred without interrupting the cyclic process data. The sub device can both receive and transmit Ethernet frames via EoE. Therefore, the standard Ethernet based protocols like HTTP and SMTP are used. This way an EtherCAT sub device can run a web server.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

1.4.3 File Access over EtherCAT (FoE)

The FoE services allow the upload and download of files to/from the sub device similar to the Trivial File Transfer Protocol (TFTP). This feature can be used to download new firmware to the sub device or to exchange large configuration data.

1.4.4 Servo profile over EtherCAT (SoE)

This mailbox protocol is used to access SERCOS identifiers in the EtherCAT sub device.

1.4.5 Vendor-specific profile over EtherCAT (VoE)

This mailbox protocol allows vendors to implement their own mailbox protocol. The Vendor and a vendor-specific protocol number identify the protocol. VoE can only be used if the main device also knows this protocol.

1.4.6 ADS over EtherCAT (AoE)

This protocol exchanges Automation Device Specification (ADS) messages over EtherCAT. Modular devices and fieldbus gateways use this protocol to embed other mailbox protocols in asynchronous messages and usually forward these messages to the intended recipient.

1.5 EtherCAT state machine

An EtherCAT sub device has different states. The main device configures the sub device and requests a state change. The sub device must check the configuration and either acknowledge or reject the state change. If there is an error, the sub device writes an error code to one of its registers.

Table 5. Sub device behavior in all EtherCAT states

State	Symbol	Configured by main device	Sub device behavior
Init	INIT	-	initial state (power up)
Pre-Operational	PreOP	Mailboxes	acyclic communication is possible
Safe-Operational	SafeOP	Input Sync Manager	send process data
Operational	OP	Output Sync Manager	process received process data
Bootstrap (optional)	BOOT	Mailboxes (Boot mode)	Boot mode (firmware update)

Not all state transitions are possible. The following figure shows all allowed transitions.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

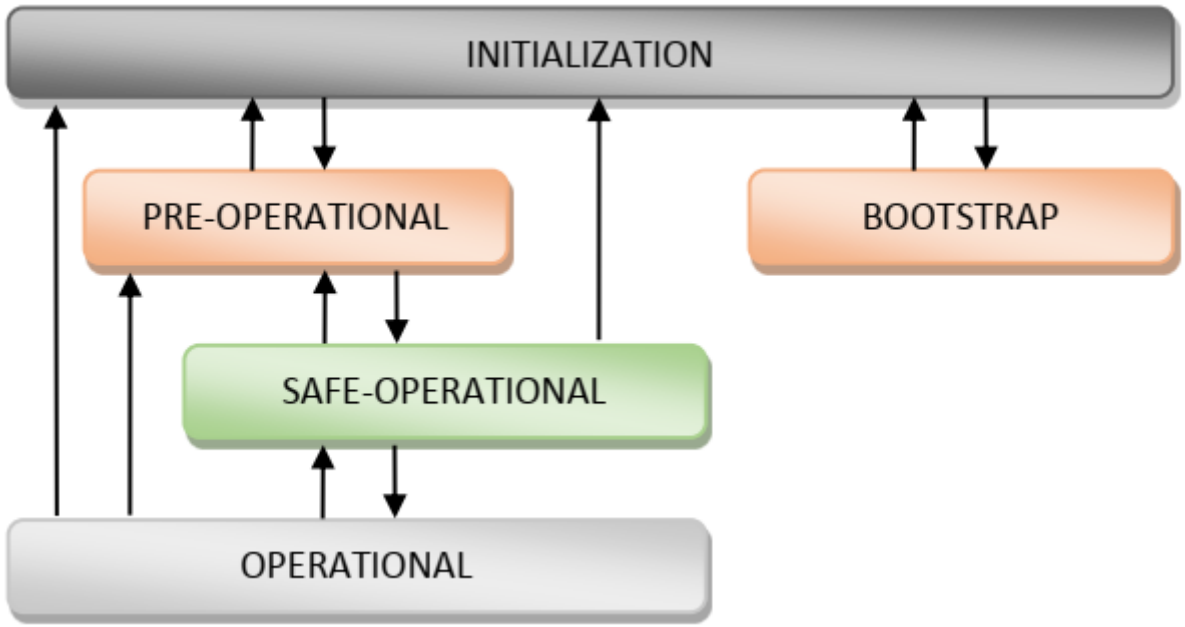

Figure 2. EtherCAT State Machine

1.6 Distributed clocks

An EtherCAT sub device Controller can support Distributed Clocks (DC). These timer units are used to synchronize all sub devices of an EtherCAT segment to a common system time. Also, one of the following behaviors can be configured.

- Generating synchronous output signals (SyncSignals)
- time stamping of input events (LatchSignals)
- interrupt generation for synchronizing the sub device application

1.7 Explicit device identification

The main device can request the sub device to put its Explicit Device ID into a special ESC register. This way the main device can explicitly identify a sub device. The Explicit Device ID must be configured in the sub device by non-EtherCAT means. For example, via rotary switches and a display.

1.8 EtherCAT subdevice information interface

The Subdevice Information Interface (SII) is an EEPROM that is directly connected to the ESC via I2C. It contains information for the ESC configuration and sub device identification. The information is used to describe the sub device's capabilities.

The Electrically Erasable Programmable Read-Only Memory (EEPROM) can be accessed via EtherCAT frames by the main device. Therefore, the main device can identify a sub device. It is also possible for the sub device to access the EEPROM via PDI when the main device grants permission.

Some ESCs, field-Programmable Gate Array (FPGA) IP cores, do not need an EEPROM. They emulate the EEPROM by issuing the read and write commands to the sub device application. The application must store the virtual EEPROM in nonvolatile memory.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

1.9 Acknowledgment by write

If an ESC IP Core is implemented into an FPGA, it can be connected to a Soft-CPU via a 32-bit memory bus. Some buses restrict access to data at the byte level. Since most ESC events are cleared by reading a register, it is possible that an ESC event is accidentally cleared without processing it. If this feature is enabled in the IP Core ESC, events can only be cleared by writing the corresponding ESC register or memory.

2 EtherCAT for GOAL

The EtherCAT library is used with GOAL, the industrial communication framework of NXP.

This chapter introduces the API that is provided to write an application for EtherCAT on GOAL. The application can also be created with Industrial Communication Creator.

The EtherCAT library is able to run several instances. An instance is a variable of the type **GOAL_ECAT_T**. Every API function requires a reference to an instance. An instance is created using the API function **goal_ecatNew**.

2.1 Public API of the EtherCAT library

This chapter gives an overview of all public functions of the EtherCAT library. There are functions that the application must call to trigger events and callbacks invoked by the library.

Note: *.Currently, the EhterCAT library is used with GOAL. The application must use the API functions defined in this chapter. It must not use the legacy API functions of previous chapters library is used with GOAL.*

2.1.1 Setting up and running the library

To initialize the library, the following function must be called *appl_init()*:

```
/*************************************************************************/
/** Application Init
 *
 * This function must initialize GOAL and all used protocol stacks.
 * Furthermore application specific resources must be initialized.
 */
GOAL_STATUS_T appl_init(
    void
)
{
    GOAL_STATUS_T res;                          /* result */

    /* initialize EtherCAT */
    res = goal_ecatInit();
    if (GOAL_RES_ERR(res)) {
       goal_logErr("Initialization of EtherCAT failed");
    }

    return res;
}
```

To create an instance of the EtherCAT library, the following function must be called:

```
/*************************************************************************/
/** Application Setup
 *
 * Setup the application.
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

```
 */
GOAL_STATUS_T appl_setup(
    void
)
{
    GOAL_STATUS_T res;                          /* result */

    res = goal_ecatNew(&pHdlEcat, GOAL_ECAT_INSTANCE_DEFAULT,
 appl_ecatCallback);
    if (GOAL_RES_ERR(res)) {
        goal_logErr("failed to create a new EtherCAT instance");
        return res;
    }

    return res;
}
```

2.1.2 Debugging

The EtherCAT library uses the GOAL logging mechanism. By default, error messages, warnings, and informational messages are printed. To enable other debugging messages, use the *GOAL* API function *goal_setDbgLevel* function.

2.1.3 Object dictionary access

The object dictionary access functions enable the application to access managed variables of the object dictionary. Application variables can be accessed directly since their variable name is known to all modules.

Table 6. OD access API

Function	Description
goal_ecatObjAddrGet()	Get a pointer to the object value
goal_ecatObjValGet()	Get the object value
goal_ecatObjValSet()	Get the object value

2.1.4 Dynamic object dictionary

Besides the static generation of the object dictionary using the design tool, an API is provided to create objects dynamically.

Table 7. OD creation API

Function	Description
goal_ecatdynOdObjAdd()	Add object
goal_ecatdynOdSubIndexAdd()	Add subindex to object
goal_ecatdynOdObjNameAdd()	Assign a name to an object
goal_ecatdynOdSubIndexNameAdd()	Assign name to subindex
goal_ecatdynOdFinish()	Finish object creation

2.1.5 Dynamic Library Configuration

These functions allow the configuration of the EtherCAT library at runtime during initialization. To take effect, they must be called before *goal_ecatNew()* is called.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 8. Dynamic configuration API

Function	Description
goal_ecatCfgEmergencyOn()	Enable CoE Emergency Service
goal_ecatCfgNumEmergencyQueueSlots()	Set size of CoE Emergency message queue
goal_ecatCfgFoeOn()	Enable FoE service
goal_ecatCfgExplDevIdOn()	Enable Explicit Device Identification
goal_ecatCfgBootstrapOn()	Enable ESM state BOOTSTRAP
goal_ecatCfgDcRequiredOn()	Make DC synchronization mandatory
goal_ecatCfgSizePdoStreamBuf()	Set size of PDO bytestream buffer
goal_ecatCfgLedStatusIndicator()	Set LED emulation to combine RUN/ERROR led

2.1.5.1 CoE API

The CoE module provides indication events to connect the application to the objects and the process data. There are also some API functions that trigger the CoE and SDO module.

Table 9. CoE indication events

Callback ID	Description
GOAL_ECAT_CB_ID_SDO_UPLOAD	Indicate SDO Read access to an object
GOAL_ECAT_CB_ID_SDO_DOWNLOAD	Check an object's new data before it is written
GOAL_ECAT_CB_ID_RxPDO_RECEIVED	Indicate reception of output process data (objects updated)
GOAL_ECAT_CB_ID_TxPDO_PREPARE	Update input process data objects before they are mapped to Sync Manager

Table 10. CoE API

Function	Description
goal_ecatEmyReqWrite()	Send a CoE emergency message

2.1.6 EoE API

There is no EoE API because when enabled, EoE internally connects to the GOAL framework.

2.1.7 FoE API

This module provides indication of events that the application must implement.

Table 11. TFOE indication events

Event	Description
GOAL_ECAT_CB_ID_FOE_READ_REQ	Open a file for reading
GOAL_ECAT_CB_ID_FOE_WRITE_REQ	Open a file for writing
GOAL_ECAT_CB_ID_FOE_READ_DATA	Get a fragment of the read file
GOAL_ECAT_CB_ID_FOE_WRITE_DATA	Write a fragment of the write file
GOAL_ECAT_CB_ID_FOE_ERROR	Error occurred, close file

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

2.1.8 EtherCAT state machine

These functions inform the application about a change in the EtherCAT state machine.

Table 12. ESM API

Function	Description
goal_ecatEsmStateGet()	Get the current ESM state

Table 13. ESM Events

Event	Description
GOAL_ECAT_CB_ID_NEW_ESM_STATE_ENTERED	Indicate new ESM state and possible error
GOAL_ECAT_CB_ID_NEW_ESM_STATE_REQUESTED	Indicate an ESM state change request
GOAL_ECAT_CB_ID_EXPLICIT_DEV_ID	Get the Explicit Device ID (during state change)

2.1.9 Data layer indication functions

These events are called by the EtherCAT library when a data layer event happens. The application programmer must implement these functions.

Table 14. Data Link Events

Function	Description
GOAL_ECAT_CB_ID_SM_WATCHDOG_EXPIRED	Process data reception timeout (Hardware watchdog)
GOAL_ECAT_CB_ID_NEW_DL_STATE	Port state change

2.1.10 Distributed clock API

If synchronization via distributed clocks is activated, the library calls the following events. The application programmer must implement them.

Table 15. DC Events

Function	Description
GOAL_ECAT_CB_ID_NEW_DC_CONFIG	Check the synchronization settings, for example, the cycle time
GOAL_ECAT_CB_ID_DC_FAIL	Indicate loss of Sync0 interrupts

3 Creating an application with GOAL

This chapter describes the steps to create an application for EtherCAT with GOAL.

3.1 Files created by the Industrial Communication Creator

The Industrial Communication Creator (ICC) must be used to specify the options for the library and to define the layout of the object dictionary.

The following source files are created:

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 16. Source code files generated by the ICC

File	Description
ecat_conf.h	Contains all configuration macros
ec_od.c	Contains the object dictionary tables
ec_objects.{c|h}	Contains the application variables of the object dictionary

These files are considered to be part of the application. Other files generated by **ICC** are ignored for this chapter.

The object dictionary has either managed variables or application variables. Managed variables are only accessible via the OD API of the library. Application variables are declared in ec_objects.{c|h} and can be used directly.

3.2 Application modules

The examples delivered with the EtherCAT library provide a good starting point for developing your own application.

Note: *Always include the header files goal_includes.h and goal_ecat.h.*

The file *goal_appl.c* demonstrates how to initialize and use the EtherCAT library in a GOAL application.

3.3 Callback mechanism

When creating an instance of the EtherCAT Library, a callback function can be assigned. The function requires the following signature:

```
/*****************************************************************************/
/** EtherCAT Callback Handler
 *
 * This function collects all callbacks from the stack and decides if the
 * callback must be handled.
 *
 * @retval GOAL_OK successful
 * @retval other failed
 */
GOAL_STATUS_T appl_ecatCallback(
    GOAL_ECAT_T *pHdlEcat,                      /**< GOAL EtherCAT handle */
    GOAL_ECAT_CB_ID_T id,                       /**< callback id */
    GOAL_ECAT_CB_DATA_T *pCb                    /**< callback parameters */
)
{
    GOAL_STATUS_T res = GOAL_OK;                /* result */

    switch (id) {
        case GOAL_ECAT_CB_ID....
            break;
    }

    return res;
}
```

The id specifies the function being called within the callback function. The following list contains all possible callback IDs:

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 17. Callback Ids

Symbol	Description
GOAL_ECAT_CB_ID_SDO_DOWNLOAD	SDO Write Access
GOAL_ECAT_CB_ID_SDO_UPLOAD	SDO Read Access
GOAL_ECAT_CB_ID_RxPDO_RECEIVED	Updated output data
GOAL_ECAT_CB_ID_TxPDO_PREPARE	Provide new input data
GOAL_ECAT_CB_ID_SYNC_FAIL	Synchronization operation failed
GOAL_ECAT_CB_ID_NEW_DL_STATE	New datalink layer status
GOAL_ECAT_CB_ID_NEW_DC_CONFIG	New DC configuration
GOAL_ECAT_CB_ID_DC_FAIL	DC Synchronization failed
GOAL_ECAT_CB_ID_SM_WATCHDOG_EXPIRED	Sync manager watchdog failed
GOAL_ECAT_CB_ID_EXPLICIT_DEV_ID	Request for explicit device id
GOAL_ECAT_CB_ID_NEW_ESM_STATE_ENTERED	New ESM state entered
GOAL_ECAT_CB_ID_NEW_ESM_STATE_REQUESTED	New ESM state requested
GOAL_ECAT_CB_ID_FOE_READ_REQ	FoE read request
GOAL_ECAT_CB_ID_FOE_READ_DATA	FoE read data
GOAL_ECAT_CB_ID_FOE_WRITE_REQ	FoE write request
GOAL_ECAT_CB_ID_FOE_WRITE_DATA	FoE write data
GOAL_ECAT_CB_ID_FOE_ERROR	FoE error
GOAL_ECAT_CB_ID_RUN_LED_STATE	RUN LED state change
GOAL_ECAT_CB_ID_EROR_LED_STATE	ERROR LED state change

Parameters are propagated to the callback function as an array of the data structure *GOAL_ECAT_CB_DATA_T*.

```
typedef union {
    uint16_t index;              /**< object index */
    uint8_t subIndex;            /**< object sub-index */
    uint8_t *pData;              /**< object data */
    uint32_t size;               /**< object size */
    GOAL_BOOL_T completeAccess;  /**< complete object is accessed */
    uint16_t dlState;            /**< new Data Link layer state */
    uint32_t dcCycleTime;        /**< current DC cycle time */
    uint32_t dcCycleTimeMin;     /**< minimum supported DC cycle time */
    uint16_t explDevId;          /**< Explicit device ID */
    uint16_t esmState;           /**< new ESM state */
    GOAL_BOOL_T esmError;        /**< device is in error state */
    uint16_t statusCode;         /**< status code explaining error */
    uint16_t appStatusCode;      /**< application specific error */
    uint32_t foePassword;        /**< password for file access */
    char *pFileName;             /**< file to be accessed via FoE */
    uint16_t foeError;           /**< FOE error code */
    uint32_t foeOffset;          /**< read/write offset in file */
    uint16_t foeMaxSize;         /**< maximum size of a FOE chunk */
    uint16_t foeActSize;         /**< actual size of a FOE chunk */
    GOAL_BOOL_T foeFinished;     /**< write access finished */
    GOAL_BOOL_T bLedEnable;      /**< led state */
} GOAL_ECAT_CB_DATA_ELEM_T;
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Depending on the callback ID, parameters and return values are provided and expected in the ID specific positions within the callback data array.

3.4 Public functions

The following section lists the public functions used in creating an application with GOAL.

3.4.1 goal_ecatInit

Purpose: Initial registration of the EtherCAT library

Function Prototype:

```
GOAL_STATUS_T goal_ecatInit(
    void
);
```

Example:

```
/*****************************************************************************/
/** Application Init
 *
 * This function must initialize GOAL and all used protocol stacks.
 * Furthermore application specific resources must be initialized.
 */
GOAL_STATUS_T appl_init(
    void
)
{
    GOAL_STATUS_T res;                          /* result */

    /* initialize EtherCAT */
    res = goal_ecatInit();
    if (GOAL_RES_ERR(res)) {
       goal_logErr("Initialization of EtherCAT failed");
    }

     return res;
}
```

Remarks

None

3.4.2 goal_ecatNew

Purpose: Create an instance of the EtherCAT library.

Function Prototype:

```
GOAL_STATUS_T goal_ecatNew(
    GOAL_ECAT_T **ppEcat,                       /**< [out] EtherCAT instance ref
 */
    const uint32_t id,                          /**< instance id */
    GOAL_ECAT_FUNC_CB_T pFunc                   /**< EtherCAT callback function
 */
);
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Example:

```
/*****************************************************************************/
/* Local variables */
/*****************************************************************************/
static GOAL_ECAT_T *pHdlEcat;                   /**< GOAL EtherCAT handle */


/*****************************************************************************/
/** Application Setup
 *
 * Setup the application.
 */
GOAL_STATUS_T appl_setup(
    void
)
{
    GOAL_STATUS_T res;                          /* result */
    goal_logInfo("create new instance");

    res = goal_ecatNew(&pHdlEcat, GOAL_ECAT_INSTANCE_DEFAULT,
 appl_ecatCallback);
    if (GOAL_RES_ERR(res)) {
        goal_logErr("failed to create a new EtherCAT instance");
        return res;
    }

    return res;
}
```

Remarks

None

3.4.3 goal_ecatCfgEmergencyOn

Purpose: Enable emergency message support.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgEmergencyOn(
    GOAL_BOOL_T enable                          /**< enable or disable feature
 */
);
```

Example:

```
/* enable CoE emergency */
res = goal_ecatCfgEmergencyOn(GOAL_TRUE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to enable CoE Emergency support");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

3.4.4 goal_ecatCfgEmergencyQueueNum

Purpose: Set a number of queue able emergency messages.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgEmergencyQueueNum(
    int16_t num                                /**< number of queue slots */
);
```

Example:

```
/* set emergency queue size to 8 */
res = goal_ecatCfgEmergencyQueueNum(8);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to set CoE Emergency Queue size to 8");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

3.4.5 goal_ecatCfgFoeOn

Purpose: Enable FoE support.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgFoeOn(
    GOAL_BOOL_T enable                         /**< enable or disable feature
 */
);
```

Example:

```
/* enable FoE */
res = goal_ecatCfgFoeOn(GOAL_TRUE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to enable FoE support");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

3.4.6 goal_ecatCfgExplDevIdOn

Purpose: Enable support for Explicit Device Identification.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgExplDevIdOn(
    GOAL_BOOL_T enable                         /**< enable or disable feature
 */
);
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Example:

```
/* enable Explicit Device Identification */
res = goal_ecatCfgExplDevIdOn(GOAL_TRUE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to enable Explicit Device Identification support");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

3.4.7 goal_ecatCfgBootstrapOn

Purpose: Enable support for BOOTSTRAP esm state.

Function prototype:

```
GOAL_STATUS_T goal_ecatCfgBootstrapOn(
    GOAL_BOOL_T enable                          /**< enable or disable feature
 */
);
```

Example:

```
/* enable BOOTSTRAP state */
res = goal_ecatCfgBootstrapOn(GOAL_TRUE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to enable BOOTSTRAP support");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

3.4.8 goal_ecatCfgDcRequiredOn

Purpose: Enable enforcement of DC mode.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgDcRequiredOn(
    GOAL_BOOL_T enable                          /**< enable or disable feature
 */
);
```

Example:

```
/* enable DC mode only */
res = goal_ecatCfgDcRequiredOn(GOAL_TRUE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to enforce DC mode");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

3.4.9 goal_ecatCfgSizePdoStreamBufSet

Purpose: Configure the PDO data buffer.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgSizePdoStreamBufSet(
    uint16_t size                               /**< size of PDO stream buffer
 */
);
```

Example:

```
/* set size of PDO buffer
 *
 * The buffer must be big enough to hold both the input and output data.
 * The size depends on the possible combinations of mappable objects.
 */
res = goal_ecatCfgSizePdoStreamBufSet(APPL_ECAT_PDO_BUF_SIZE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to set PDO buffer size to %d", APPL_ECAT_PDO_BUF_SIZE);
}
```

Remarks:

This function must be called before *goal_ecatNew*.

To use the maximum process data length for the CCM module, the buffer must be configured to 2 * 68 = 136 bytes.

3.4.10 goal_ecatCfgLedStatusIndicator

Purpose: Configure the behavior of LED emulation.

Function Prototype:

```
GOAL_STATUS_T goal_ecatCfgLedStatusIndicator(
    GOAL_BOOL_T enable                          /**< enable or disable feature
 */
);
```

Example:

```
/* configure dual LED status indicator */
res = goal_ecatCfgLedStatusIndicator(GOAL_FALSE);
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to configure LED status indicator");
}
```

Remarks:

This function must be called before *goal_ecatNew*.

If set to FALSE, the LED emulation supports a dedicated RUN and ERROR indicator. If set to TRUE, LED emulation supports a combined STATUS indicator.

3.4.11 goal_ecatObjAddrGet

Purpose: Get a pointer to the object value.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Function Prototype:

```
GOAL_STATUS_T goal_ecatObjAddrGet(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint8_t subIndex,                           /**< object sub-index */
    uint8_t **ppData,                           /**< [out] object data pointer
 reference */
    uint32_t *pSize                             /**< [out] object size reference
 */
);
```

Example:

```
GOAL_STATUS_T res;                          /* return */
uint8_t errorRegister;                      /* variable */
uint32_t size;                              /* variable size */

res = goal_ecatObjAddrGet(pEcat, 0x1001, 0x0, &errorRegister, &size);
if (GOAL_RES_OK(res)) {
    goal_logInfo("error register contains %d" errorRegister);
}
```

Remarks:

3.4.12 goal_ecatObjValGet

Purpose: get the object value.

Function Prototype:

```
goal_ecatObjValGet(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint8_t subIndex,                           /**< object sub-index */
    uint8_t *pData,                             /**< [out] object data buffer */
    uint32_t *pSize                             /**< [out] object size reference
 */
);
```

Example:

```
GOAL_STATUS_T res;                          /* return */
uint32_t value = 0;                         /* variable value */
uint32_t size = 0;                          /* variable size */

res = goal_ecatObjValGet(pHdlEcat, 0x1001, 0, (uint8_t *) &value, &size);
if (GOAL_RES_ERR(res)) {
    return res;
}
```

Remarks:

None

3.4.13 goal_ecatObjValSet

Purpose: Set the object value.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Function Prototype:

```
GOAL_STATUS_T goal_ecatObjValSet(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint8_t subIndex,                           /**< object sub-index */
    uint8_t *pData,                             /**< [in] new object value */
    uint32_t size                               /**< size of new data */
);
```

Example:

```
res = goal_ecatObjValSet(
        pHdlEcat,
        0x1008,
        0x00,
        (uint8_t *) APPL_ECAT_PRODUCT_NAME,
        GOAL_STRLEN(APPL_ECAT_PRODUCT_NAME));
if (GOAL_RES_ERR(res)) {
    goal_logErr("failed to set product name in object dictionary");
    return res;
}
```

Remarks:

None

3.4.14 goal_ecatdynOdObjAdd

Purpose: Add object.

Function Prototype:

```
GOAL_STATUS_T goal_ecatdynOdObjAdd(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint16_t objTpye,                           /**< object type (VAR, ARRAY,
 RECORD) */
    uint16_t dataType                           /**< data type of object */
);
```

Example:

```
res = goal_ecatdynOdObjAdd(
    pHdlEcat,
    0x1000,
    GOAL_ECAT_OBJCODE_VAR,
    GOAL_ECAT_DATATYPE_UNSIGNED32);
```

Remarks:

None

3.4.15 goal_ecatdynOdSubIndexAdd

Purpose: Add a subindex to the object.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Function Prototype:

```
GOAL_STATUS_T goal_ecatdynOdSubIndexAdd(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint8_t subIndex,                           /**< object sub-index */
    uint16_t dataType,                          /**< data type of object */
    uint16_t attr,                              /**< attributes (R/W/
mappable,...) */
    uint8_t *pDefVal,                           /**< default value of entry */
    uint8_t *pMinVal,                           /**< minimum value of entry */
    uint8_t *pMaxVal,                           /**< maximum value of entry */
    uint32_t size,                              /**< size of non-numerical
 objects */
    uint8_t *pVar                               /**< application variable */
);
```

Example:

```
uint32ValueMin = 0x0;
uint32ValueDef = 0x00030191;
uint32ValueMax = 0xFFFFFFFF;

res = goal_ecatdynOdSubIndexAdd(
    pHdlEcat,
    0x1000,
    0x00,
    GOAL_ECAT_DATATYPE_UNSIGNED32,
    EC_OBJATTR_RD | EC_OBJATTR_MAN | EC_OBJATTR_NUMERIC | EC_OBJATTR_NO_LIMITS,
    (uint8_t *) &uint32ValueDef,
    (uint8_t *) &uint32ValueMin,
    (uint8_t *) &uint32ValueMax,
    4,
    NULL);
```

Remarks:

None

3.4.16 goal_ecatdynOdObjNameAdd

Purpose: Assign a name to the object.

Function Prototype:

```
GOAL_STATUS_T goal_ecatdynOdObjNameAdd(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    char *pName                                 /**< name of object */
);
```

Example:

```
res = goal_ecatdynOdObjNameAdd(pHdlEcat, 0x1000, "Device Type");
```

Remarks:

None

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

3.4.17 goal_ecatdynOdSubIndexNameAdd

Purpose: Assign a name to the subindex.

Function Prototype:

```
GOAL_STATUS_T goal_ecatdynOdSubIndexNameAdd(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t index,                             /**< object index */
    uint8_t subIndex,                           /**< object sub-index */
    char *pName                                 /**< name of sub-index */
);
```

Example:

```
res = goal_ecatdynOdSubIndexNameAdd(pHdlEcat, 0x1018, 1, "Vendor Id");
```

Remarks:

None

3.4.18 goal_ecatdynOdFinish

Purpose: Finish object creation.

Function Prototype:

```
GOAL_STATUS_T goal_ecatdynOdFinish(
    GOAL_ECAT_T *pEcat                          /**< EtherCAT instance data */
);
```

Example:

```
/* finish object dictionary creation */
if (GOAL_RES_OK(res)) {
    res = goal_ecatdynOdFinish(pHdlEcat);
}
```

Remarks:

This function ends dynamic creation of objects. It must be called after the last object was created. After calling this function, no additional objects can be created.

3.4.19 goal_ecatEsmStateGet

Purpose: Get the current ESM state.

Function Prototype:

```
GOAL_STATUS_T goal_ecatEsmStateGet(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint8_t *pState                             /**< [out] esm state */
);
```

Example:

```
GOAL_STATUS_T res;                          /* return */
uint8_t state;                              /* EtherCAT state */
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

```
/* check if state is BOOTSTRAP */
res = goal_ecatEsmStateGet(pHdlEcat, &state);
if (GOAL_RES_OK(res) && GOAL_ECAT_ESM_STATE_BOOTSTRAP == state) {
}
```

Remarks:

None

3.4.20 goal_ecatEmcyReqWrite

Purpose: Generate an emergency message for transmission.

Function Prototype:

```
GOAL_STATUS_T goal_ecatEmcyReqWrite(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t errorCode,                         /**< standard error code */
    uint8_t *pManuErr                           /**< manufacturer error code */
);
```

Example:

```
GOAL_STATUS_T res;                          /* result */
uint8_t errData[5];                         /* dummy error data */

errData[0] = 0x01;
errData[1] = 0x02;
errData[2] = 0x03;
errData[3] = 0x04;
errData[4] = 0x05;

res = goal_ecatEmcyReqWrite(
    pEcat,
    APPL_ECAT_EMCY_DEMO_ERROR_CODE,
    errData);
```

Remarks:

None

3.4.21 goal_ecatGetVersion

Purpose: Get the version information about the EtherCAT library.

Function Prototype:

```
GOAL_STATUS_T goal_ecatGetVersion(
    char *strVersion,                           /**< [out] stack version */
    int size                                    /**< buffer size */
);
```

Example:

```
char version[10];

res = goal_ecatGetVersion(version, 10);
```


All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Remarks:

None

3.4.22 goal_ecatEsmLocalErrorSet

Purpose: Set error from application

Function Prototype:

```
GOAL_STATUS_T goal_ecatEsmLocalErrorSet(
    GOAL_ECAT_T *pEcat,                         /**< EtherCAT instance data */
    uint16_t errorCode                          /**< new AL status code */
);
```

Example:

```
/* set local error */
goal_ecatEsmLocalErrorSet(pEcat, 0x8000);
```

Remarks:

None

3.5 Definitions

Table 18. Application specific SDO accesses errors

Symbol	Description
GOAL_ERR_SDO_HARDWARE	Hardware error
GOAL_ERR_SDO_TOO_LOW	Value too low
GOAL_ERR_SDO_TOO_HIGH	Value too high
GOAL_ERR_SDO_APPL	Application-specific reason
GOAL_ERR_SDO_DEV_STATE	Current device state insufficient
GOAL_ERR_SDO_GENERAL	Unspecified SDO error

Table 19. Object dictionary access errors

Symbol	Description
GOAL_ERR_OBJ_NOT_FOUND	Object not found
GOAL_ERR_SIDX_NOT_FOUND	Subindex not found
GOAL_ERR_OBJ_VAL_TO_HIGH	The new value exceeds the upper limit of the object
GOAL_ERR_OBJ_VAL_TO_LOW	The new value exceeds the lower limit of the object
GOAL_ERR_OBJ_SIZE	Invalid data size for object

Table 20. AL status codes

Symbol	Description
GOAL_ECAT_STATE_CODE_OK	No error

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 20. AL status codes...*continued*

Symbol	Description
GOAL_ECAT_STATE_CODE_UNSPEC	Unspecified error
GOAL_ECAT_STATE_CODE_NO_MEM	No memory
GOAL_ECAT_STATE_CODE_INVALID_SETUP	Invalid device set-up
GOAL_ECAT_STATE_CODE_SII_MISMATCH	SII/EEPROM information does not match firmware
GOAL_ECAT_STATE_CODE_FW_UPDATE_FAIL	Firmware updates not successful, old firmware still running
GOAL_ECAT_STATE_CODE_LICENSE_ERR	License error
GOAL_ECAT_STATE_CODE_INVALID_REQ	Invalid requested state change
GOAL_ECAT_STATE_CODE_UNKNOWN_REQ	Unknown requested state
GOAL_ECAT_STATE_CODE_NO_BOOTSTRAP	Bootstrap not supported
GOAL_ECAT_STATE_CODE_NO_FIRMWARE	No valid firmware
GOAL_ECAT_STATE_CODE_INVALID_MBX_CONF_B	Invalid mailbox configuration
GOAL_ECAT_STATE_CODE_INVALID_MBX_CONF	Invalid mailbox configuration
GOAL_ECAT_STATE_CODE_INVALID_SYNCM_CONFIG	Invalid sync manager configuration
GOAL_ECAT_STATE_CODE_NO_INPUTS	No valid inputs available
GOAL_ECAT_STATE_CODE_NO_OUTPUTS	No valid outputs
GOAL_ECAT_STATE_CODE_UNSPECIFIC_SYNC_ERROR	Synchronization error
GOAL_ECAT_STATE_CODE_SYNCM_WATCHDOG	Sync manager watchdog
GOAL_ECAT_STATE_CODE_INVALID_SYNCM_TYPE	Invalid sync manager types
GOAL_ECAT_STATE_CODE_INVALID_OUTPUT_CONFIG	Invalid output configuration
GOAL_ECAT_STATE_CODE_INVALID_INPUT_CONFIG	Invalid input configuration
GOAL_ECAT_STATE_CODE_INVALID_WD_CONFIG	Invalid watchdog configuration
GOAL_ECAT_STATE_CODE_COLD_START	Sub device needs a cold start
GOAL_ECAT_STATE_CODE_NEED_INIT	Sub device needs INIT
GOAL_ECAT_STATE_CODE_NEED_PREOP	Sub device needs PREOP
GOAL_ECAT_STATE_CODE_NEED_SAFEOP	Sub device needs SAFEOP
GOAL_ECAT_STATE_CODE_INVALID_INPUT_MAPPING	Invalid input mapping
GOAL_ECAT_STATE_CODE_INVALID_OUTPUT_MAPPING	Invalid output mapping
GOAL_ECAT_STATE_CODE_INVALID_SETTING	Inconsistent settings
GOAL_ECAT_STATE_CODE_NO_FREERUN	FreeRun not supported
GOAL_ECAT_STATE_CODE_INVALID_SYNC_MODE	SyncMode not supported
GOAL_ECAT_STATE_CODE_FREERUN_3BUF	FreeRun needs 3-Buffer Mode
GOAL_ECAT_STATE_CODE_BACKGROUND_WD	Background watchdog
GOAL_ECAT_STATE_CODE_INVALID_INPUT_OUTPUT	No valid inputs and outputs
GOAL_ECAT_STATE_CODE_FATAL_SYNC_ERROR	Fatal sync error
GOAL_ECAT_STATE_CODE_NO_SYNC_RECEIVED	No sync error
GOAL_ECAT_STATE_CODE_SMALL_CYCLE_TIME	Cycle time too small

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 20. AL status codes...*continued*

Symbol	Description
GOAL_ECAT_STATE_CODE_INVALID_DC_SYNC	Invalid DC SYNC configuration
GOAL_ECAT_STATE_CODE_INVALID_DC_LATCH	Invalid DC latch configuration
GOAL_ECAT_STATE_CODE_PLL_ERR	PLL error
GOAL_ECAT_STATE_CODE_UNSYNCED_IO	DC sync IO error
GOAL_ECAT_STATE_CODE_UNSYNCED	DC sync timeout error
GOAL_ECAT_STATE_CODE_INVALID_SYNC_CYCLE_TIME	DC invalid sync cycle time
GOAL_ECAT_STATE_CODE_DC_SYNC_0_CYCLE_TIME	Sync0 cycle time
GOAL_ECAT_STATE_CODE_DC_SYNC_1_CYCLE_TIME	DC sync1 cycle time
GOAL_ECAT_STATE_CODE_MBX_AOE	MBX_AOE
GOAL_ECAT_STATE_CODE_MBX_EOE	MBX_EOE
GOAL_ECAT_STATE_CODE_MBX_COE	MBX_COE
GOAL_ECAT_STATE_CODE_MBX_FOE	MBX_FOE
GOAL_ECAT_STATE_CODE_MBX_SOE	MBX_SOE
GOAL_ECAT_STATE_CODE_MBX_VOE	MBX_VOE
GOAL_ECAT_STATE_CODE_EEPROM_NO_ACCESS	EEPROM no access
GOAL_ECAT_STATE_CODE_EEPROM_ERR	EEPROM Error
GOAL_ECAT_STATE_CODE_EXT_HW_ERR	External hardware not ready
GOAL_ECAT_STATE_CODE_RESTARTED	Sub device restarted locally
GOAL_ECAT_STATE_CODE_DEVICE_ID_UPDATE	Device identification value updated
GOAL_ECAT_STATE_CODE_MODULE_IDENT_MISMATCH	Detected module identification list does not match
GOAL_ECAT_STATE_CODE_APP_AVAILABLE	Application controller available

Table 21. EtherCAT State Machine states

Symbol	Description
GOAL_ECAT_ESM_STATE_UNKNOWN	Unknown ESM state
GOAL_ECAT_ESM_STATE_INIT	ESM state INIT
GOAL_ECAT_ESM_STATE_PREOP	ESM state Pre-OP
GOAL_ECAT_ESM_STATE_BOOTSTRAP	ESM state BOOTSTRAP
GOAL_ECAT_ESM_STATE_SAFEOP	ESM state SafeOP
GOAL_ECAT_ESM_STATE_OP	ESM state OP

Table 22. FoE error codes

Symbol	Description
GOAL_ECAT_FOE_ERR_NOT_DEFINED	Undefined error
GOAL_ECAT_FOE_ERR_NOT_FOUND	Not found

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 22. FoE error codes...*continued*

Symbol	Description
GOAL_ECAT_FOE_ERR_ACCESS_DENIED	Access denied
GOAL_ECAT_FOE_ERR_DISK_FULL	Disk full
GOAL_ECAT_FOE_ERR_ILLEGAL	Illegal
GOAL_ECAT_FOE_ERR_PACKET_NUMBER_WRONG	Wrong packet number
GOAL_ECAT_FOE_ERR_ALREADY_EXISTS	Exists
GOAL_ECAT_FOE_ERR_NO_USER	No user
GOAL_ECAT_FOE_ERR_BOOTSTRAP_ONLY	Bootstrap only
GOAL_ECAT_FOE_ERR_NOT_BOOTSTRAP	Not Bootstrap
GOAL_ECAT_FOE_ERR_NO_RIGHTS	No rights
GOAL_ECAT_FOE_ERR_PROGRAM_ERROR	Program error
GOAL_ECAT_FOE_ERR_CSUM_WRONG	Checksum wrong
GOAL_ECAT_FOE_ERR_FIRMWARE	Wrong firmware
GOAL_ECAT_FOE_ERR_NO_FILE_TO_READ	No file to read
GOAL_ECAT_FOE_ERR_NO_HEADER	The file header does not exist
GOAL_ECAT_FOE_ERR_FLASH	Error
GOAL_ECAT_FOE_ERR_FILE_INCOMPATIBLE	File incompatible
GOAL_ECAT_FOE_BUSY_STATE	The server is busy

Table 23. Object dictionary object types

Symbol	Description
GOAL_ECAT_OBJCODE_VAR	Object is a variable object
GOAL_ECAT_OBJCODE_ARRAY	Object is an array object
GOAL_ECAT_OBJCODE_RECORD	An object is a recorded object

Table 24. Object dictionary data types

Symbol	Description
GOAL_ECAT_DATATYPE_UNKNOWN	Data type: UNKNOWN
GOAL_ECAT_DATATYPE_BOOL	Data type: BOOL
GOAL_ECAT_DATATYPE_INTEGER8	Data type: INTEGER8
GOAL_ECAT_DATATYPE_INTEGER16	Data type: INTEGER16
GOAL_ECAT_DATATYPE_INTEGER32	Data type: INTEGER32
GOAL_ECAT_DATATYPE_UNSIGNED8	Data type: UNSIGNED8
GOAL_ECAT_DATATYPE_UNSIGNED16	Data type: UNSIGNED16
GOAL_ECAT_DATATYPE_UNSIGNED32	Data type: UNSIGNED32
GOAL_ECAT_DATATYPE_REAL32	Data type: REAL32

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 24. Object dictionary data types...*continued*

Symbol	Description
GOAL_ECAT_DATATYPE_VISSTRING	Data type: VISSTRING
GOAL_ECAT_DATATYPE_OCTETSTRING	Data type: OCTETSTRING
GOAL_ECAT_DATATYPE_REAL64	Data type: REAL64
GOAL_ECAT_DATATYPE_INTEGER64	Data type: INTEGER64

4 Creating an application with more than four sync managers

Since Version 1.8.0 the EtherCAT library supports more than 4 Sync Managers.

This chapter describes the necessary steps that must be performed in the Industrial Communication Creator (ICC) to support more than 4 Sync Managers.

Usually, the following Sync Manager Configuration is used (and recommended by the specification).

Sync Manager	Type	Description
0	MBoxOut	Mailbox (main device to sub device)
1	MBoxIn	Mailbox (sub device to main device)
2	Outputs	Process Data (main device to sub device)
3	Inputs	Process Data (sub device to main device)

4.1 Creating more sync managers

Using Sync Manager 0 and Sync Manager 1 as Mailboxes remains mandatory. Configure the remaining Sync Managers freely as needed.

To configure Sync Managers N for process data transfer, execute the following steps in the ICC:

1. Create Subindex N + 1 for Object 0x1C00 (Sync Manager Communication Type).
2. Select the communication type (input or output).
3. Configure a start address in the Process Data RAM of the ESC. Process Data Sync Managers occupy a RAM area three times their size.
4. Ensure that the output triggers the Sync Manager watchdog.
5. Create a PDO Assignment Object for Sync Manager (0x1C10 + N).
   - Define which PDOs are assigned to the Sync Manager.
6. Create the Sync Manager Synchronization Object (0x1C30 + N).
   - Modify the default value of Subindex 2 (Synchronization Type):
     - Use 0x01 for outputs or inputs that do not belong to an output.
     - Use 0x20 + Sync Manager Number of the corresponding output for inputs that belong to an output.

4.2 Defining sync units

It is possible to restrict the usage of PDOs to a Sync Unit (a pair of Output and Input) in the ICC. Follow these steps:

1. Select the corresponding PDO Assignment Object of a Sync Manager (0x1C10 + N).
2. Enable the **Select Sync Unit** option.
3. Enter a Sync Unit ID.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

4. Assign another Sync Manager to the same Sync Unit ID.
5. For each desired PDO-Mapping Object (0x1600 to 0x17FF and 0x1A00 to 0x1BFF):
   a. Enable the **Select Sync Unit** option.
   b. Enter the corresponding Sync Unit ID.

5 EtherCAT main device PLC - CODESYS

The CODESYS software from https://de.codesys.com/ is a powerful and simple-to-use choice for running PLC software without special hardware requirements. Therefore, it can be easily used for establishing a network communication with the GOAL network stacks.

Note: *PORT registers as a vendor to add EtherCAT devices and will migrate these licenses to NXP in due course.*

5.1 Requirements

- GOAL device under test (dut)
- https://de.codesys.com/ V3.5 SP16 Patch 4
- Windows PC
- https://npcap.com/ v1.31

CODESYS example Project

This CODESYS PLC project is designed to work with GOAL EtherCAT application 01_simple_io.

5.2 Initial setup and configuration

1. Open the Windows properties for the interface of choice and disable all options except *NPCAP*.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

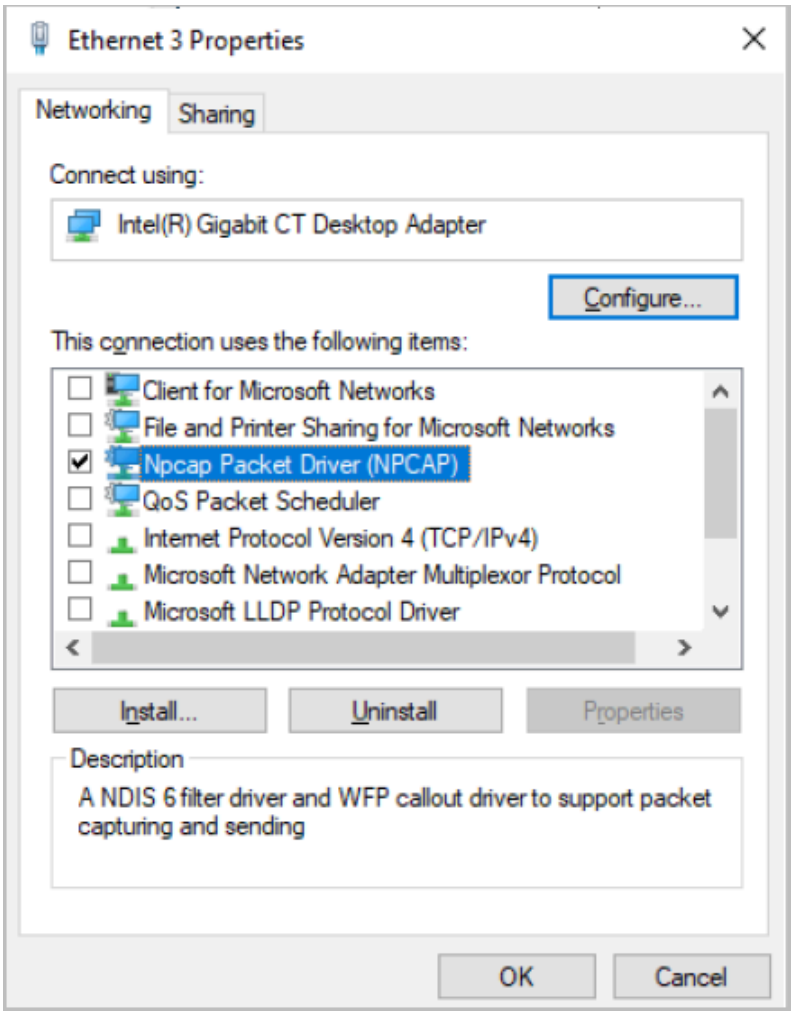

Figure 3. Windows properties

2. Run *CODESYS Control Win V3 - x64 SysTray* and choose *Run PLC* from the symbol menu.
3. Run *CODESYS Control Win V3 - x64* and choose *Run Gateway* from the symbol menu. The red element on the symbol signs active software.

Figure 4. Gateway and PLC in RUN mode

4. Start *CODESYS V3.5 SP16 Patch 4* and open the provided example project from above.
5. Choose **Tools > Device Repository**.
6. Install the xml file of the GOAL device, which is at the platform specific *edt* directory at *projects/goal_ecat*/ <sample project>/.* Afterward, it is available in the list of Installed Device Descriptions.

© 2025 NXP B.V. All rights reserved.

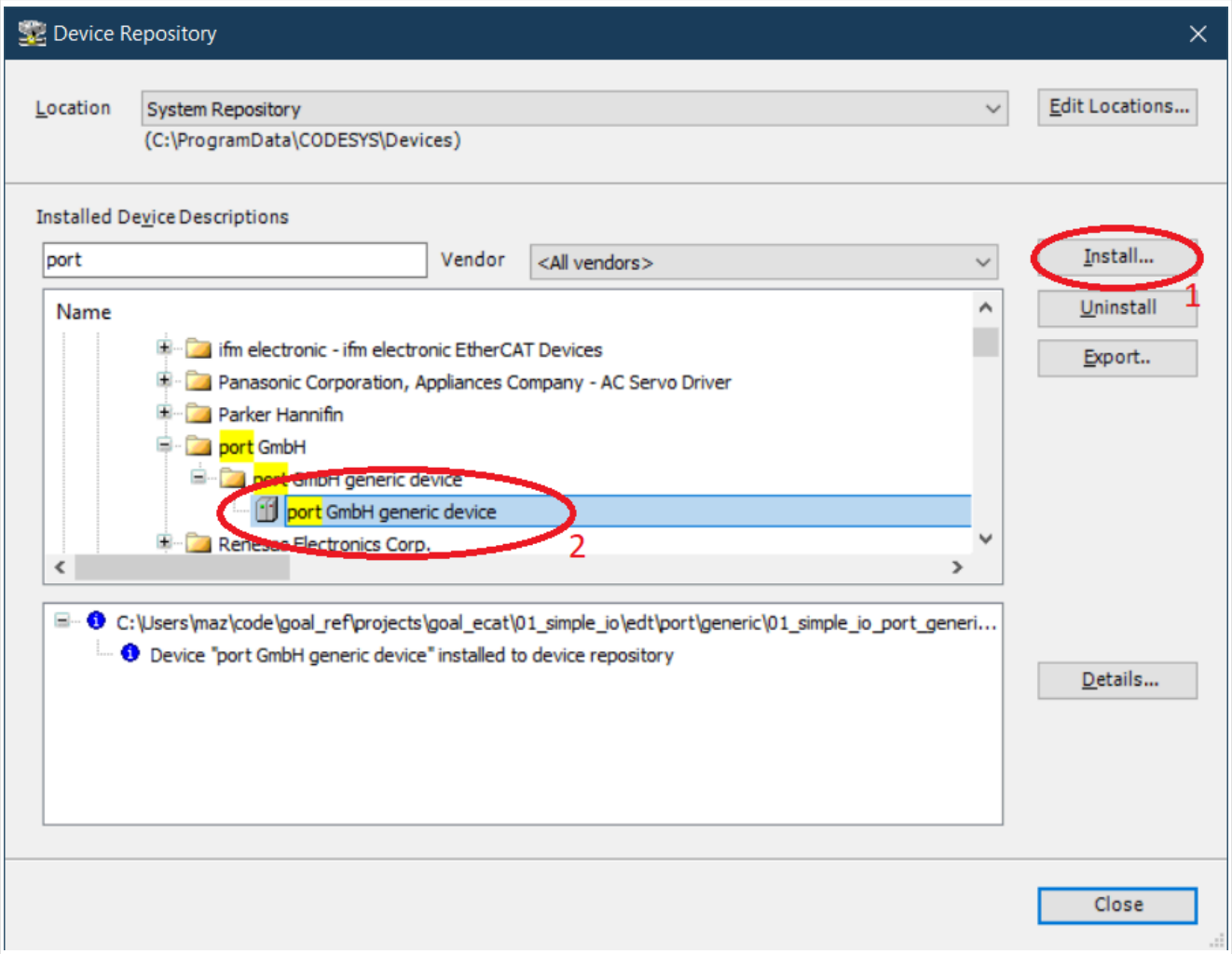

Figure 5. Device repository

7. Choose device configuration and Scan Network for detecting the local PLC.
8. After confirming the controller by OK, the state switches from grey to green.

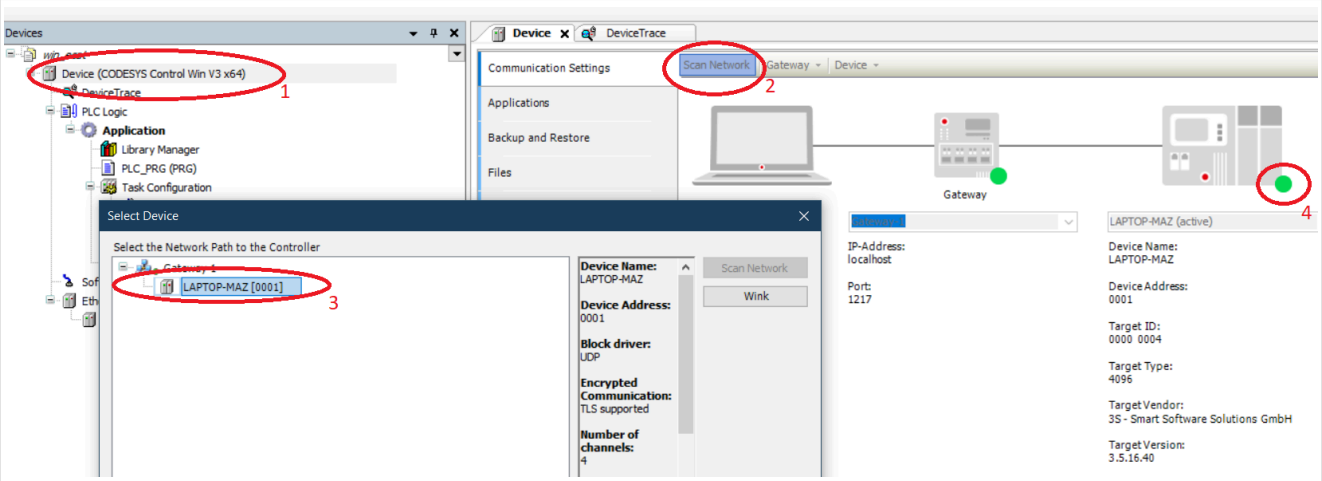

Figure 6. Choose device configuration

9. Open EtherCAT_main device configuration.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

10. *Browse…* the network port on your local machine.

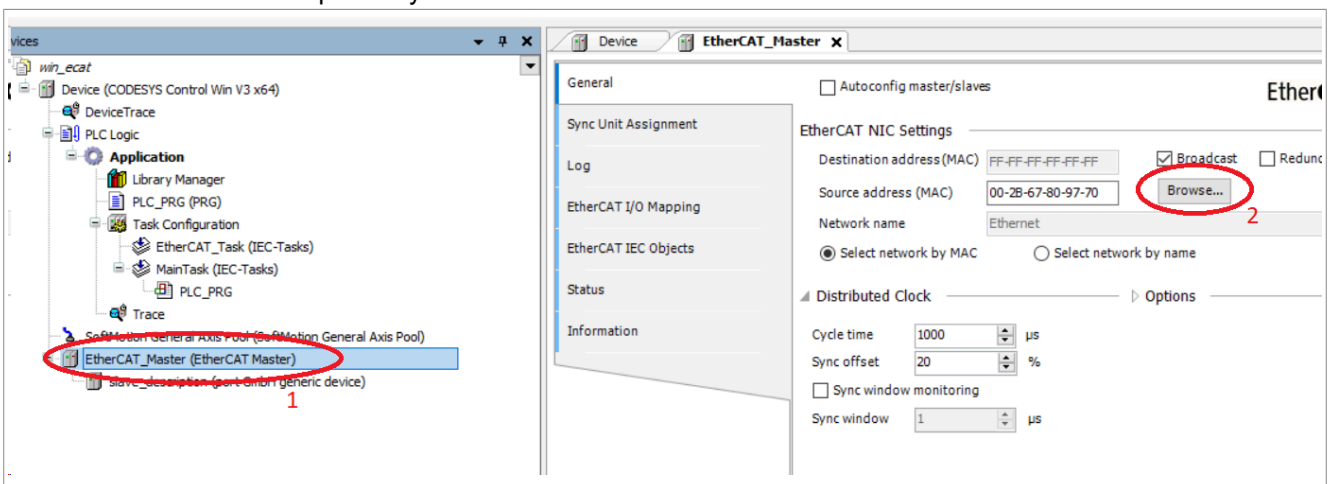

Figure 7. Select the network port

11. Update the sub device by searching the previously imported xml description in the list
12. Confirm via *Update Device*.

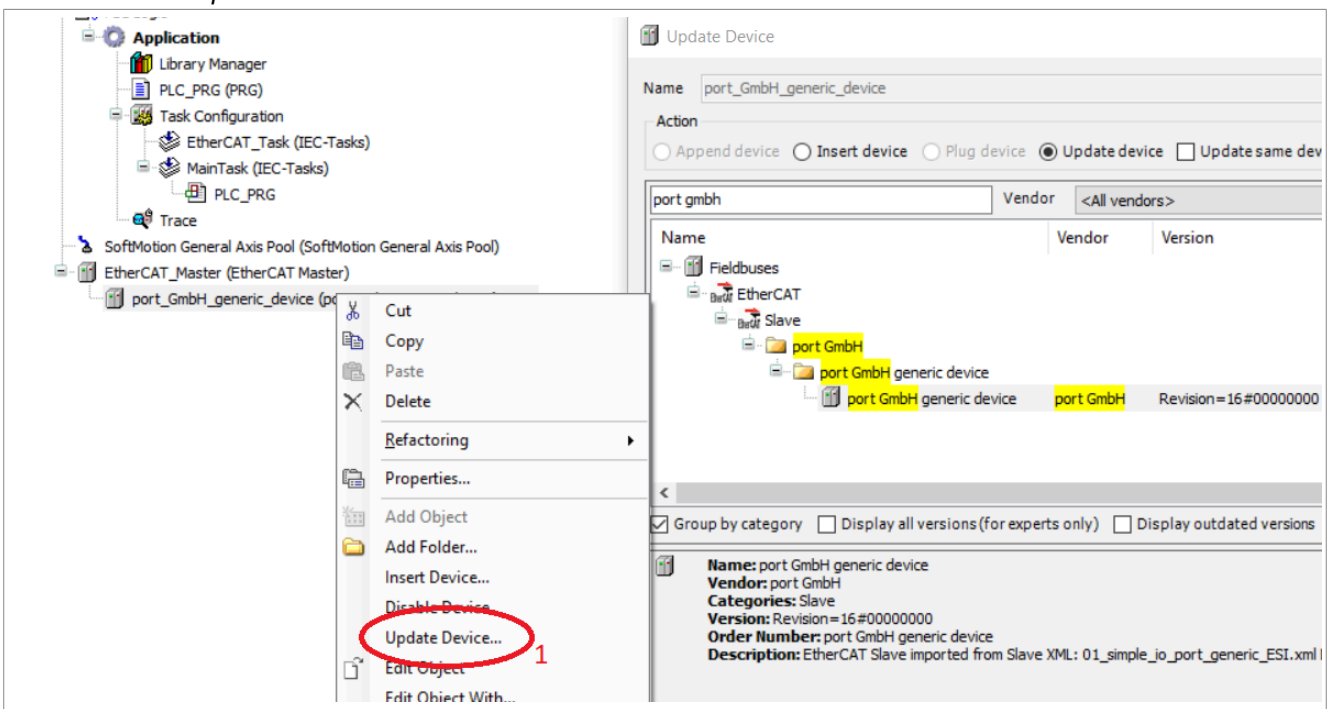

Figure 8. Update device

13. Open the Sub device configuration.
14. Choose EtherCAT I/O Mapping and assign PLC variables to IOs as listed in the table below.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

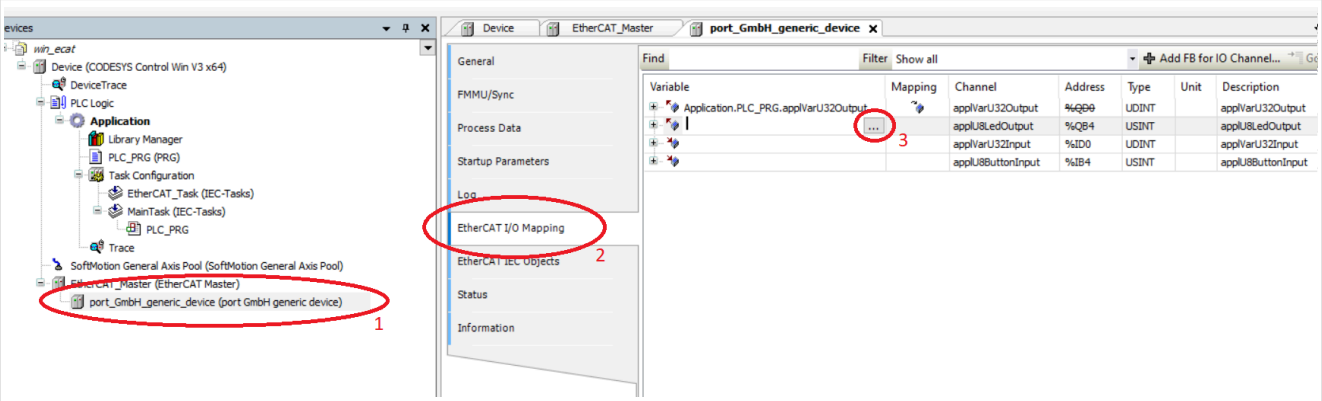

Figure 9. Choose EtherCAT I/O Mapping and assign PLC variables to IOs

Table 25. EtherCAT I/O Mapping to assign PLC variables to IOs

Variable	Description
Application.PLC_PRG.applVarU32Output	applVarU32Output
Application.PLC_PRG.applVarU8Output	applU8LedOutput
Application.PLC_PRG.applVarU32Input	applVarU32Input
Application.PLC_PRG.applVarU8Input	applU8ButtonInput

5.2.1 Enabling distributed clocks (optional)

1. Open the **General** tab in the sub device configuration.
2. Select **Expert settings**.
3. Confirm that the distributed clock is enabled and the selected DC is *DC-Synchron.*

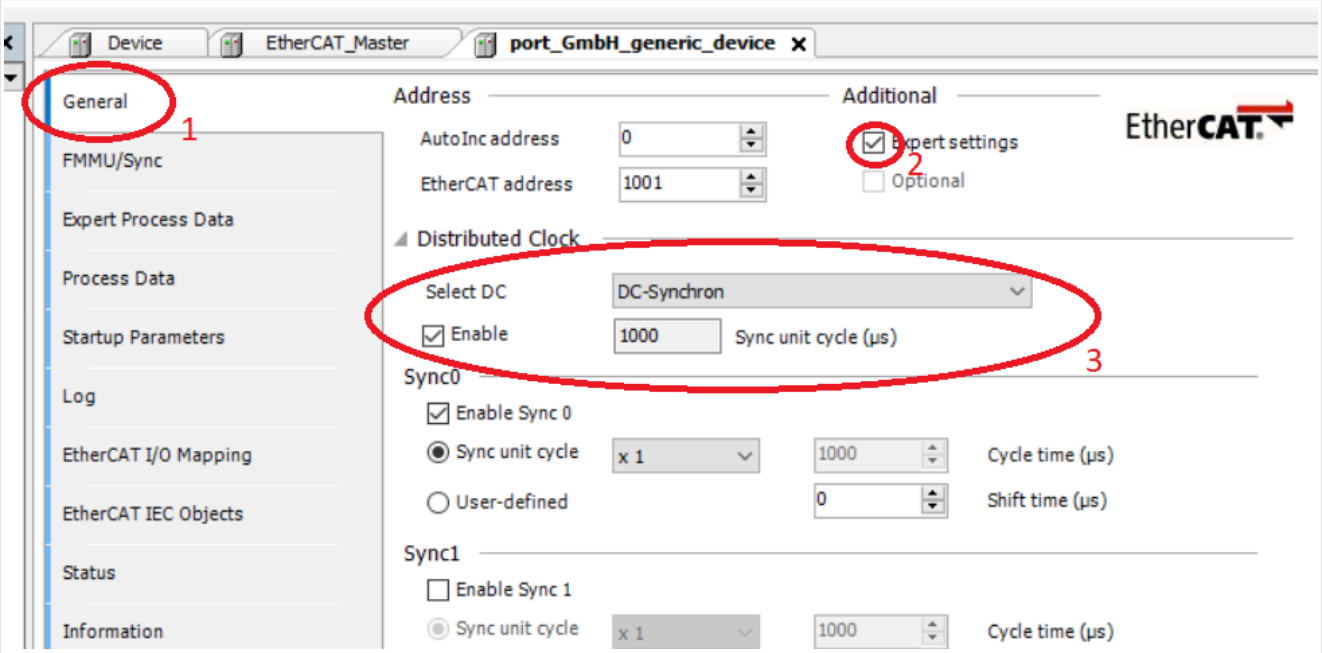

Figure 10. Distributed clock

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

5.3 Running the PLC software

1. Build the PLC software.
2. Log in afterward.

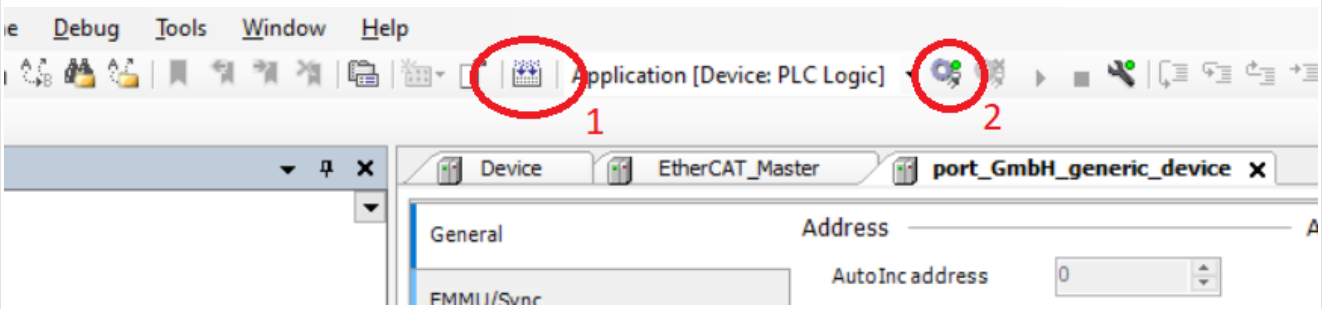

Figure 11. Build and login

3. Confirm the request about performing a download.

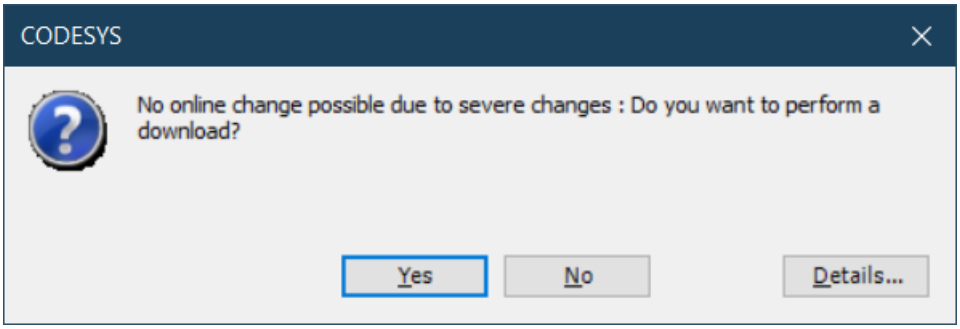

Figure 12. Confirm the download

4. If the PLC program is already running, use the login with download option.

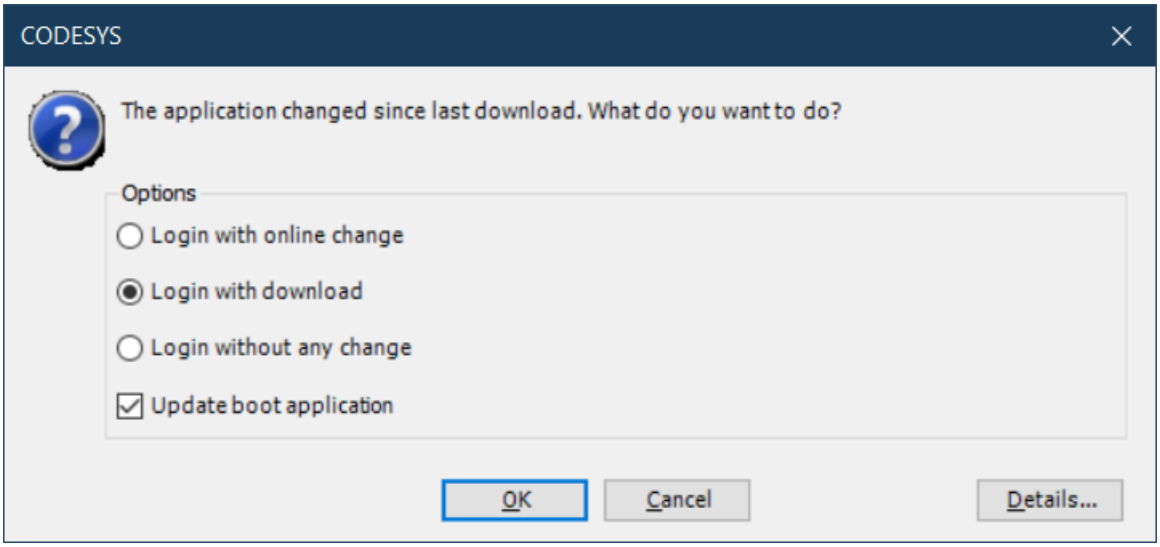

Figure 13. Login with download

5. Open the PLC_PRG.
6. Start PLC_PRG by clicking the left symbol. The connection is established, exchanging application data.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

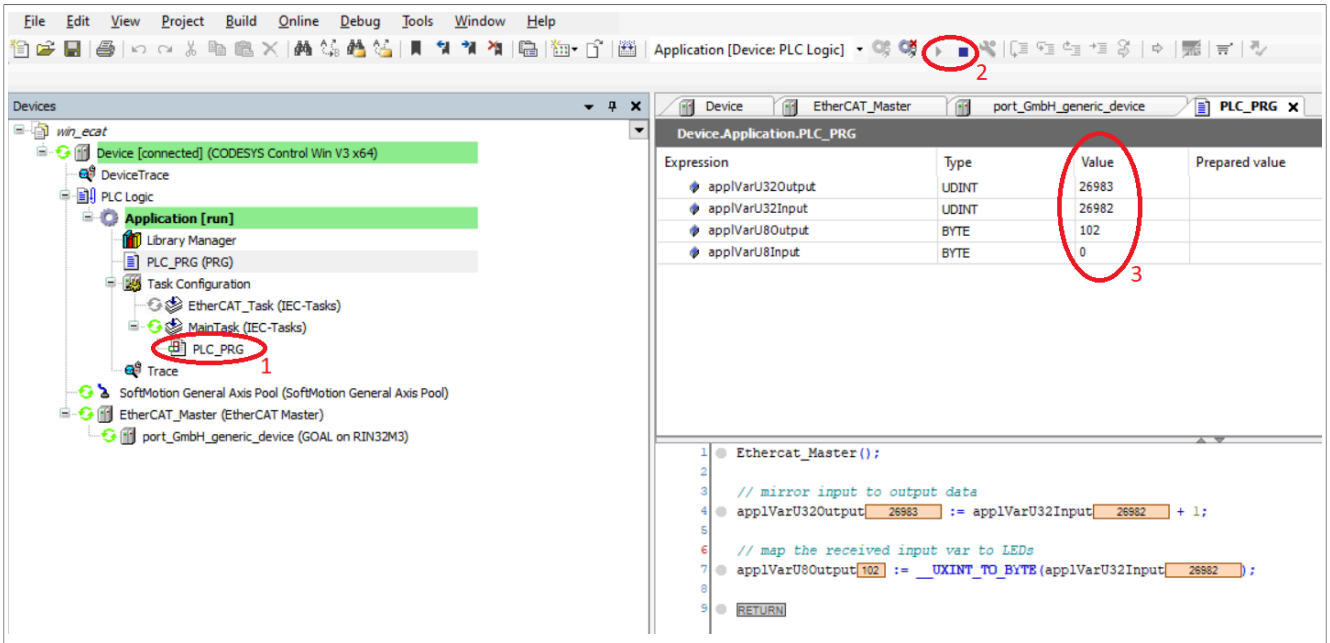

Figure 14. Establish the connection

6 Acronyms

Table 26 lists the acronyms used in this document along with their description.

Table 26. Acronyms

Acronym	Description
ADS	Automation Device Specification
AoE	ADS over EtherCAT
API	Application Programming Interface
ASIC	Application-Specific Integrated Circuit
BSP	Board Support Package
CAN	Controller Area Network
CANopen	CAN-based fieldbus protocol
CCM	Cyclic Communication Module
CoE	CANopen over EtherCAT
DC	Distributed Clocks
DLL	Data Link Layer
EoE	Ethernet over EtherCAT
EEPROM	Electrically Erasable Programmable Read-Only Memory
ESC	EtherCAT Sub device Controller
ESI	EtherCAT Slave Information
FoE	File Access over EtherCAT
FPGA	Field-Programmable Gate Array

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Table 26. Acronyms...*continued*

Acronym	Description
FSoE	Fail Safe over EtherCAT
GOAL	Industrial Communication Framework by Port
HAL	Hardware Abstraction Layer
ICC	Industrial Communication Creator
IP	Intellectual Property (core)
MAC	Media Access Control
MDP	Modular Device Profile
MII	Media Independent Interface
NMT	Network Management (state machine)
OD	Object Dictionary
OP	Operational (EtherCAT state)
PDO	Process Data Object
PDI	Process Data Interface
PLC	Programmable Logic Controller
PreOP	Pre-Operational (EtherCAT state)
RAM	Random Access Memory
RxPDO	Receive Process Data Object
SafeOP	Safe-Operational (EtherCAT state)
SDO	Service Data Object
SII	Subdevice Information Interface
SM	Sync Manager
SoE	Servo profile over EtherCAT
SPI	Serial Peripheral Interface
TxPDO	Transmit Process Data Object
VoE	Vendor-specific profile over EtherCAT

7 Note about the source code in the document

Example code shown in this document has the following copyright and BSD-3-Clause license:

Copyright 2025 NXP Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials must be provided with the distribution.
3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

8 Revision history

Table 27 summarizes the revisions to this document.

Table 27. Revision history

Document ID	Release date	Description
UM12403 v.3.0	12 December 2025	Initial public release

© 2025 NXP B.V. All rights reserved.

Legal information

Definitions

Draft — A draft status on a document indicates that the content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included in a draft version of a document and shall have no liability for the consequences of use of such information.

Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale — NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at https://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Suitability for use in non-automotive qualified products — Unless this document expressly states that this specific NXP Semiconductors product is automotive qualified, the product is not suitable for automotive use. It is neither qualified nor tested in accordance with automotive testing or application requirements. NXP Semiconductors accepts no liability for inclusion and/or use of non-automotive qualified products in automotive equipment or applications.

In the event that customer uses the product for design-in and use in automotive applications to automotive specifications and standards, customer (a) shall use the product without NXP Semiconductors' warranty of the product for such automotive applications, use and specifications, and (b) whenever customer uses the product for automotive applications beyond NXP Semiconductors' specifications such use shall be solely at customer's own risk, and (c) customer fully indemnifies NXP Semiconductors for any liability, damages or failed product claims resulting from customer design and use of the product for automotive applications beyond NXP Semiconductors' standard warranty and NXP Semiconductors' product specifications.

HTML publications — An HTML version, if available, of this document is provided as a courtesy. Definitive information is contained in the applicable document in PDF format. If there is a discrepancy between the HTML document and the PDF document, the PDF document has priority.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. — NXP B.V. is not an operating company and it does not distribute or sell products.

Trademarks

Notice: All referenced brands, product names, service names, and trademarks are the property of their respective owners.

NXP — wordmark and logo are trademarks of NXP B.V.

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Microsoft, Azure, and ThreadX — are trademarks of the Microsoft group of companies.

© 2025 NXP B.V. All rights reserved.

Tables

Tab. 1. Sync Manager Usage 2
Tab. 2. CoE object dictionary 3
Tab. 3. Table: PDO-mapping objects 3
Tab. 4. Sync Manager Assignment objects 3
Tab. 5. Sub device behavior in all EtherCAT states 5
Tab. 6. OD access API 8
Tab. 7. OD creation API 8
Tab. 8. Dynamic configuration API 9
Tab. 9. CoE indication events 9
Tab. 10. CoE API 9
Tab. 11. TFOE indication events 9
Tab. 12. ESM API 10
Tab. 13. ESM Events 10
Tab. 14. Data Link Events 10
Tab. 15. DC Events 10
Tab. 16. Source code files generated by the ICC 11
Tab. 17. Callback Ids 12
Tab. 18. Application specific SDO accesses errors 23
Tab. 19. Object dictionary access errors 23
Tab. 20. AL status codes 23
Tab. 21. EtherCAT State Machine states 25
Tab. 22. FoE error codes 25
Tab. 23. Object dictionary object types 26
Tab. 24. Object dictionary data types 26
Tab. 25. EtherCAT I/O Mapping to assign PLC variables to IOs 32
Tab. 26. Acronyms 34
Tab. 27. Revision history 36

Figures

Fig. 1. DO Mapping in EtherCAT 4
Fig. 2. EtherCAT State Machine 6
Fig. 3. Windows properties 29
Fig. 4. Gateway and PLC in RUN mode 29
Fig. 5. Device repository 30
Fig. 6. Choose device configuration 30
Fig. 7. Select the network port 31
Fig. 8. Update device 31
Fig. 9. Choose EtherCAT I/O Mapping and assign PLC variables to IOs 32
Fig. 10. Distributed clock 32
Fig. 11. Build and login 33
Fig. 12. Confirm the download 33
Fig. 13. Login with download 33
Fig. 14. Establish the connection 34

All information provided in this document is subject to legal disclaimers.

© 2025 NXP B.V. All rights reserved.

Contents

1 Introduction 2
1.1 Supported platforms 2
1.2 EtherCAT sub device controller 2
1.3 Sync managers 2
1.4 Mailbox protocols 3
1.4.1 CANopen over EtherCAT (CoE) 3
1.4.2 Ethernet over EtherCAT (EoE) 4
1.4.3 File Access over EtherCAT (FoE) 5
1.4.4 Servo profile over EtherCAT (SoE) 5
1.4.5 Vendor-specific profile over EtherCAT (VoE) 5
1.4.6 ADS over EtherCAT (AoE) 5
1.5 EtherCAT state machine 5
1.6 Distributed clocks 6
1.7 Explicit device identification 6
1.8 EtherCAT subdevice information interface 6
1.9 Acknowledgment by write 7
2 EtherCAT for GOAL 7
2.1 Public API of the EtherCAT library 7
2.1.1 Setting up and running the library 7
2.1.2 Debugging 8
2.1.3 Object dictionary access 8
2.1.4 Dynamic object dictionary 8
2.1.5 Dynamic Library Configuration 8
2.1.5.1 CoE API 9
2.1.6 EoE API 9
2.1.7 FoE API 9
2.1.8 EtherCAT state machine 10
2.1.9 Data layer indication functions 10
2.1.10 Distributed clock API 10
3 Creating an application with GOAL 10
3.1 Files created by the Industrial Communication Creator 10
3.2 Application modules 11
3.3 Callback mechanism 11
3.4 Public functions 13
3.4.1 goal_ecatInit 13
3.4.2 goal_ecatNew 13
3.4.3 goal_ecatCfgEmergencyOn 14
3.4.4 goal_ecatCfgEmergencyQueueNum 15
3.4.5 goal_ecatCfgFoeOn 15
3.4.6 goal_ecatCfgExplDevIdOn 15
3.4.7 goal_ecatCfgBootstrapOn 16
3.4.8 goal_ecatCfgDcRequiredOn 16
3.4.9 goal_ecatCfgSizePdoStreamBufSet 17
3.4.10 goal_ecatCfgLedStatusIndicator 17
3.4.11 goal_ecatObjAddrGet 17
3.4.12 goal_ecatObjValGet 18
3.4.13 goal_ecatObjValSet 18
3.4.14 goal_ecatdynOdObjAdd 19
3.4.15 goal_ecatdynOdSubIndexAdd 19
3.4.16 goal_ecatdynOdObjNameAdd 20
3.4.17 goal_ecatdynOdSubIndexNameAdd 21
3.4.18 goal_ecatdynOdFinish 21
3.4.19 goal_ecatEsmStateGet 21
3.4.20 goal_ecatEmcyReqWrite 22
3.4.21 goal_ecatGetVersion 22
3.4.22 goal_ecatEsmLocalErrorSet 23
3.5 Definitions 23
4 Creating an application with more than four sync managers 27
4.1 Creating more sync managers 27
4.2 Defining sync units 27
5 EtherCAT main device PLC - CODESYS 28
5.1 Requirements 28
5.2 Initial setup and configuration 28
5.2.1 Enabling distributed clocks (optional) 32
5.3 Running the PLC software 33
6 Acronyms 34
7 Note about the source code in the document 35
8 Revision history 36
Legal information 37

Please be aware that important notices concerning this document and the product(s) described herein, have been included in section 'Legal information'.

© 2025 NXP B.V. All rights reserved.